EEP3020: Assignment

On

"CAN Protocol"

By
Dudhani Omkar Sanjay(B21EE017)
Abhaymani Singh(B21EE001)
Bikas Singh(B21EE015)

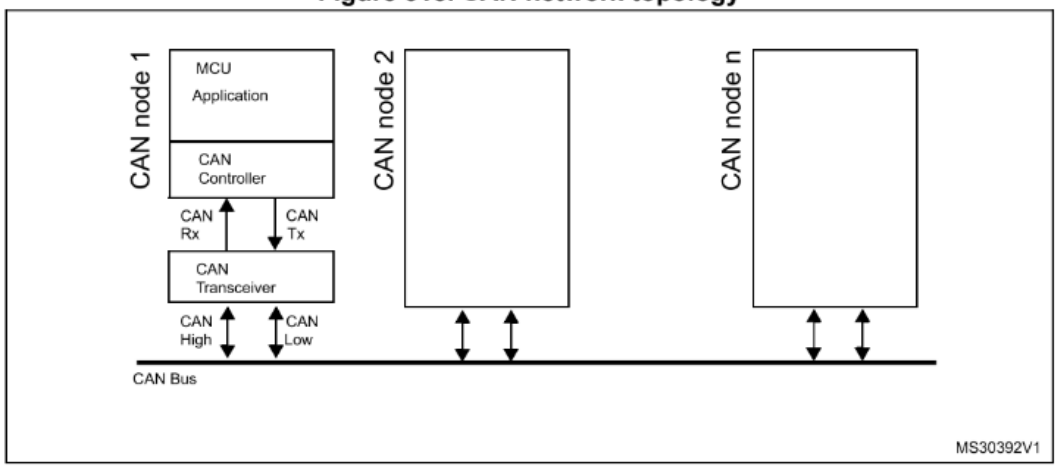
OVERVIEW AND PURPOSE

- The Controller Area Network (CAN) is a communication protocol that enables microcontrollers and devices within a system to communicate with each other without a host computer. It was originally developed by Bosch in the mid-1980s to address the need for a robust and reliable method of communication in automotive applications. Since then, it has found widespread use in various industries beyond automotive, including industrial automation, aerospace, medical devices, and more.

- CAN is designed to facilitate communication between electronic control units (ECUs) or nodes in a network, allowing them to exchange data and control information in real-time. It enables distributed intelligence, where individual nodes can perform specific functions autonomously while cooperating with other nodes on the network.

TOPOLOGY OF CAN NETWORK

Figure 318. CAN network topology



OVERVIEW OF bxCAN MODULE:

bxCAN refers to the CAN (Controller Area Network) controller module found in microcontrollers produced by STMicroelectronics, a leading semiconductor manufacturer.

The bxCAN module provides hardware support for implementing the CAN communication protocol, allowing microcontrollers to communicate with other devices or nodes on a CAN bus. It includes features such as message transmission and reception, message filtering, error detection, and more.

FEATURES:

BxCAN controller module boasts a range of powerful features designed to facilitate robust and reliable communication in a variety of application.

Following are the features summarized:

- Protocol Support: Supports CAN protocol version 2.0 A, B Active.
- Bit Rates: Supports bit rates up to 1 Mbit/s.
- Time-Triggered Communication: Supports Time Triggered Communication option.
- Transmission: Three transmit mailboxes, configurable transmit priority, time stamp on SOF transmission.
- Reception: Two receive FIFOs with three stages, scalable filter banks, identifier list feature, configurable FIFO overrun, time stamp on SOF reception.
- Time-Triggered Communication Option: Supports time-triggered communication.
- Retransmission Control: Disable automatic retransmission mode.
- Timer: 16-bit free running timer, time stamp sent in last two data bytes.
- Management: Maskable interrupts, software-efficient mailbox mapping at a unique address space.

REGISTERS USED:

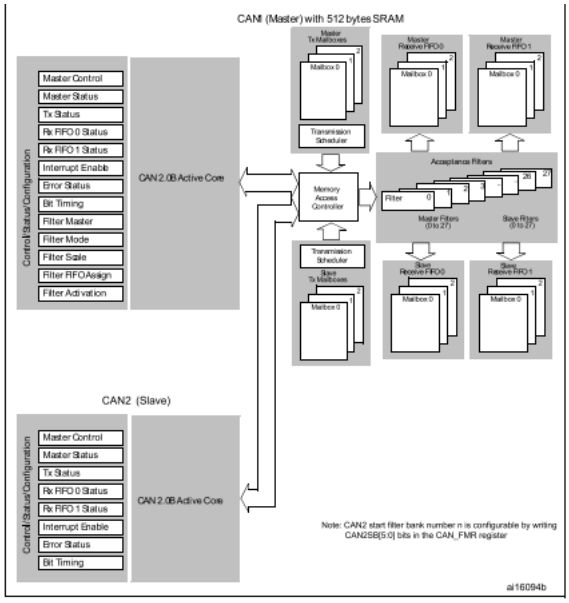
- CONTROL REGISTER
- STATUS REGISTER
- CONFIGURATION REGISTER

The application uses these registers to:
-Configure CAN parameters, e.g. baud rate
-Request transmissions
-Handle receptions
-Manage interrupts
-Get diagnostic information

WORKING:

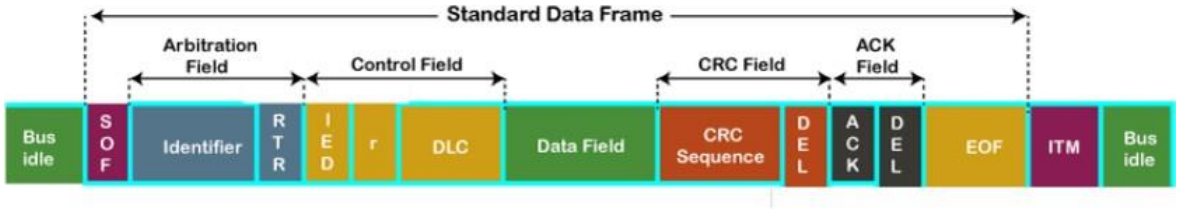
- Three transmit mailboxes are provided to the software for setting up messages. The transmission Scheduler decides which mailbox has to be transmitted first.

- Two receive FIFOs are used by hardware to store the incoming messages. Three complete messages can be stored in each FIFO(First In First Out). The FIFOs are managed completely by hardware.



DUAL CAN BlOCK DIAGRAM

Can Message Structure :



- CAN messages comprise an identifier (ID), data length code (DLC), data bytes, and a cyclic redundancy check (CRC).

- The ID serves to uniquely identify the message and dictates its priority on the bus.

- DLC indicates the number of data bytes in the message, which can range up to 8 bytes.

- Data bytes contain the actual information being transmitted.

- The CRC functions for error detection, ensuring message integrity throughout transmission.

IMPLEMENTATION OF TEST MODES :

Implementation of modes such as loop and silent typically involves configuring the behavior of a CAN (Controller Area Network) controller in a microcontroller or embedded system.

1. Loopback Mode:
   - In loopback mode, transmitted messages are internally looped back to the CAN controller's receive buffer without being transmitted onto the bus.
   - This mode is useful for testing and debugging purposes, allowing the microcontroller to verify the integrity of transmitted messages without affecting other nodes on the CAN bus. To implement loopback mode, you need to set the appropriate configuration bits or registers in the CAN controller.
   - When in loopback mode, the microcontroller will transmit messages as usual, but instead of being sent onto the bus, they are received internally

by the CAN controller for verification.

2. Silent Mode:

- In silent mode, the microcontroller's CAN controller behaves as if it were transmitting messages onto the bus, but it does not actually drive the CANH and CANL lines.
- This mode allows the microcontroller to monitor bus activity without actively participating in communication.
- Silent mode is often used in scenarios where the microcontroller needs to listen to CAN bus traffic without interfering with other nodes or causing disruption.
- Implementation of silent mode involves configuring the CAN controller to enable silent operation, typically through specific configuration registers or settings.
- When in silent mode, the microcontroller's CAN controller will receive messages from the bus and process them internally without transmitting any messages onto the bus.

Loop back combined with silent mode

Loopback combined with silent mode is a configuration in CAN (Controller Area Network) controllers where both loopback mode and silent mode are enabled simultaneously. In this configuration, the CAN controller operates in a mode where transmitted messages are internally looped back for testing and debugging purposes, while also silently monitoring bus activity without actively participating in communication.

Code:

Header .h file:

```c
#include "stm32f4xx.h"
#include<stdio.h>
#include<string.h>
#include <stdint.h>
#include <limits.h>


typedef struct message{
  int len;
  uint32_t msg;
  int valid;

}message;


//FIFO implementation for reciever

typedef struct FIFO {
    int top;
    unsigned capacity;
    message* array;
} FIFO;
```

```c
typedef struct TX_mailbox{
  message * tx_msg;
  int state;
}TX_mailbox;


typedef struct RX_mailbox{
    int state;
    FIFO * rx_box;
}RX_mailbox;

  typedef struct
{
    uint32_t ID;
    uint32_t DLC;
    uint32_t CAN_MCR;
    uint32_t CAN_MSR;
    uint32_t CAN_TSR;
    uint32_t CAN_RF0R;
    uint32_t CAN_RF1R;
    uint32_t CAN_IER;
    uint32_t CAN_ESR;
    uint32_t CAN_BTR;
    uint32_t CAN_TI0R;
    TX_mailbox outbox[3];
    RX_mailbox inbox[2];
}CAN_ref;
```

Explanation:

- The Header file contains the struct definitions for the CAN controller
- There are 5 structs defined as above :
    - Message: Contains transmission data frame in bitform also two indicators for length and message validity which are set by hardware.
    - FIFO: This is a stack implementation for storing message data types. It is used in receiver mailboxes.
    - TX_mailbox and RX_mailbox are respectively transmission and receiver mailbox declarations.
    - CAN_ref is a declaration for the CAN controller. It contains all the registers and mailboxes for functioning.

Main.c file:

- Main.c file contains all the function implementation for bxCAN specification except error handling and filter programs for simplicity.
- CAN_init Function :
  - This function initializes the CAN controller and assigns the memory to all the registers.
  - All register values are default initialized to their respective reset values.
  -
```c
void CAN_init(CAN_ref * CAN){
    CAN->ID = 0x3; //specified as per policy
    //CAN->DLC =
    CAN->CAN_MCR = 0X0;
    CAN->CAN_MSR = (1<<11) | (1<<10) | (1<<1);
    CAN->CAN_TSR = (7<<26);
    CAN->CAN_RF0R = 0x0;
    CAN->CAN_RF1R = 0x0;
    CAN->CAN_IER = 0x0;
    CAN->CAN_ESR = 0x0;
    CAN->CAN_BTR = (1<<21) | (3<<16);
    CAN->CAN_TI0R = 0x0;
    transmit_mailbox_setup(CAN);
    recieve_mailbox_setup(CAN);
}
```

- CAN_reset Function:
  - This function resets all the components of the CAN controller registers.
  -
```c
// Function for restting CAN
void CAN_reset(CAN_ref * CAN){
    if(CAN->CAN_MCR && 1<<15){
      CAN->CAN_MCR = 0X0;
      CAN->CAN_MSR = (1<<11) | (1<<10) | (1<<1);
      CAN->CAN_TSR = (7<<26);
      CAN->CAN_RF0R = 0x0;
      CAN->CAN_RF1R = 0x0;
      CAN->CAN_IER = 0x0;
      CAN->CAN_ESR = 0x0;
      CAN->CAN_BTR = (1<<21) | (3<<16);
      CAN->CAN_TI0R = 0x0;
    }
}
```

- transmit_mailbox_setup Function:
  - This function allocates the memory and initiates the transmitter-side mailboxes.

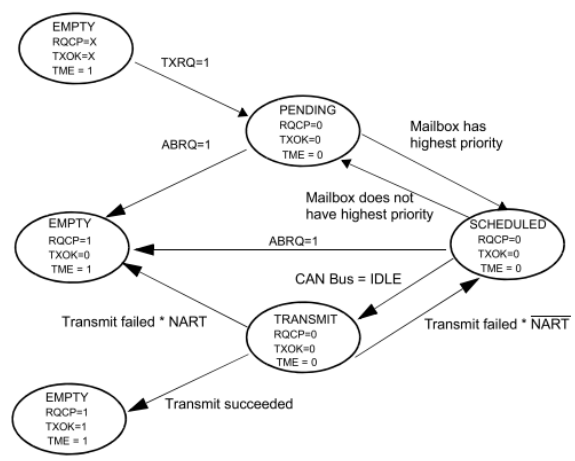
```c
void recieve_mailbox_setup(CAN_ref * CAN){
  for (int i = 0;i<3;i++){
    CAN->inbox[i] = *(RX_mailbox*)malloc(sizeof(RX_mailbox));
    CAN->inbox[i].state = 0;
  }

}
```

- recive_mailbox_setup Function:
  - This function allocates the memory and initiates the receiver-side mailboxes.
  -
```
void recieve_mailbox_setup(CAN_ref * CAN){
  for (int i = 0;i<3;i++){
    CAN->inbox[i] = *(RX_mailbox*)malloc(sizeof(RX_mailbox));
    CAN->inbox[i].state = 0;
  }

}
```

- transmit_push Function:
  - This function implements transmission handling as per the bxCAN specifications.
  - It implements the following state diagram:

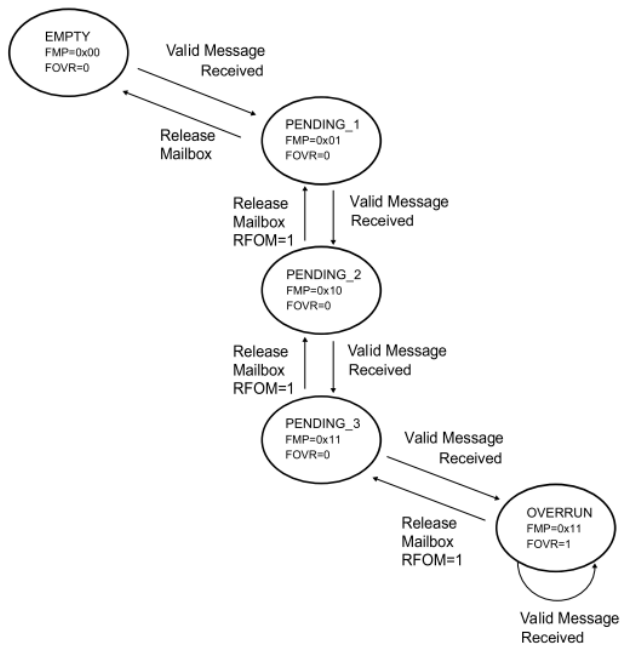
```
void transmit_push(CAN_ref * CAN,message *msg){

  if(CAN->CAN_TSR && !(1<<0)){
    CAN->outbox[0].tx_msg = msg;
    CAN->outbox[0].state = 1;
    CAN->outbox[0].state = 2;
    while(!(CAN->CAN_TSR && 1<<7) && !(CAN->CAN_TSR && 1<<1));
    Transmit_protocol(CAN->outbox[0].tx_msg);
    CAN->CAN_TSR |= (1<<0);
    CAN->outbox[0].state = 0;
  }
  else if(CAN->CAN_TSR && (1<<8)){
    CAN->outbox[1].tx_msg = msg;
    CAN->outbox[1].state = 1;
    while(!(CAN->CAN_TSR && (1<<0)));
    CAN->outbox[1].state = 2;
    while(!(CAN->CAN_TSR && 1<<15) && !(CAN->CAN_TSR && 1<<9));
    Transmit_protocol(CAN->outbox[1].tx_msg);
    CAN->CAN_TSR |= (1<<8);
    CAN->outbox[1].state = 0;
  }
  else if(CAN->CAN_TSR && (1<<16)){
    CAN->outbox[2].tx_msg = msg;
    CAN->outbox[2].state = 1;
    while(!(CAN->CAN_TSR && (1<<0)) && (!(CAN->CAN_TSR && (1<<8)));
    CAN->outbox[2].state = 2;
    while(!(CAN->CAN_TSR && 1<<23) && !(CAN->CAN_TSR && 1<<17));
    Transmit_protocol(CAN->outbox[2].tx_msg);
    CAN->CAN_TSR |= (1<<16);
    CAN->outbox[2].state = 0;
  }
}
```

Code



State Diagram

- Activate_recieve_message_mailbox Function:
  - This function implements receiver mailbox handling as per the bxCAN specifications.
  - It implements the following state diagram:



Code:

```
void activate_recieve_message_box0(CAN_ref * CAN){
  while(1){
    message msg = RX(CAN);
    if(CAN->CAN_RF0R && (1<<4) && !isFull(CAN->inbox[0].rx_box)){
      if(CAN->inbox[0].rx_box->top == 1){
        CAN->CAN_RF0R |= 0x1;
        CAN->CAN_RF0R &= !(0x8);
      }
      else{
        CAN->CAN_RF0R &= !(0xB);
      }
    }
    if(msg.valid && !(CAN->CAN_RF0R && (1<<4))){
      push(CAN->inbox[0].rx_box,msg);
      CAN->inbox[0].state += 1;
      CAN->CAN_RF0R += 1;
    }
    if(CAN->CAN_RF0R && (1<<5)){
      pop(CAN->inbox[0].rx_box);
      CAN->inbox[0].state -= 1;
      CAN->CAN_RF0R += 1;
    }
  }
}
```

References:

Can resource